



Automation Testing Assignment

MAAL WALLET AUTOMATION TESTING REPORT

Course: Quality Assurance and Software Testing

Project: Maal Wallet System

Prepared By:

Group Members	ID
Salman Ali	UGR/7808/14
Fuad Mohammed	UGR/6052/14
Emran Yonas	UGR/1649/14
Beka Birhanu	UGR/3402/14

Submission Date: May 24, 2026

1. INTRODUCTION

Maal Wallet is a financial wallet platform that provides services such as user onboarding, wallet management, peer-to-peer transfers, agent cash-in and cash-out operations, KYC verification, BNPL financing, disputes, notifications, reporting, and administrative management.

This report documents the automation testing framework and testing approach designed for the Maal Wallet backend system.

The automation suite follows a **black-box end-to-end API testing methodology**. Instead of testing isolated endpoints only, the framework models realistic business workflows involving multiple actors such as:

- Customers
- Agents
- Administrators
- Accountants
- Anonymous users

This approach is highly suitable for financial systems because wallet behavior depends on:

- Authentication and authorization
- OTP verification
- Stateful business workflows
- Idempotency rules
- Notifications and asynchronous behavior
- Financial transaction integrity

The report is based on the provided assignment instructions and the scripts located under the **Test scripts/** directory.

2. CHOSEN AUTOMATION FRAMEWORK

2.1 Selected Framework

The selected automation framework is a **custom Node.js end-to-end API automation framework** implemented using ECMAScript modules.

Core Technologies Used

The framework includes the following technologies and components:

- Native Node.js **.mjs** modules
- **fetch**-based API requests
- OpenAPI/Swagger contract discovery
- DAG-based orchestration
- Assertion helper libraries
- OTP/SMS validation helpers
- TOTP generation for MFA verification
- WebSocket realtime testing
- Structured JSON/text logging
- Secret redaction support

Main Implementation Files

Component	Location
Runner	Test scripts/e2e/run.mjs
Orchestrator	Test scripts/e2e/orchestrator.mjs
Context	Test scripts/e2e/context.mjs
Libraries	Test scripts/e2e/lib/
Flow Modules	Test scripts/e2e/flows/
Validation Runner	Test scripts/validation/backend-validation-runner.mjs
Load Testing	Test scripts/load/ws-realtime-load.mjs

2.2 Named Automation Method

The scripting approach used in this project is named:

Maal Flow-DAG Automation Method

The framework follows six major stages:

1. Discover

Discover API contracts and verify system readiness.

2. Prepare

Prepare users, tokens, devices, OTPs, fixtures, and idempotency keys.

3. Execute

Execute business flows as DAG modules with dependency ordering.

4. Assert

Validate status codes, business behavior, authorization rules, and side effects.

5. Measure

Measure endpoint coverage against the OpenAPI specification.

6. Report

Generate structured logs and reporting artifacts.

2.3 Why the Framework Is Suitable

The chosen framework is highly suitable for Maal Wallet because the system's primary risks are business-rule and backend-related risks rather than only browser UI risks.

A financial wallet system must validate:

- Money movement integrity
- Authentication and authorization
- OTP and MFA workflows
- KYC approval processes
- Transaction reversals
- Audit logging
- Notifications
- Idempotency protections

These requirements are better validated using deterministic API-level end-to-end testing.

Key Benefits

- Realistic business-flow testing
- Multi-role actor testing
- Positive and negative scenario coverage
- Reusable test logic
- Stateful workflow execution
- Controlled test data generation
- Built-in idempotency support
- Async event verification
- Security-aware logging
- CI/CD compatibility

2.4 Learning Resources Consulted

The following resources supported the implementation approach:

- Assignment instructions ([assignment.md](#))
- Internal E2E README documentation
- OpenAPI and Swagger concepts
- Node.js ECMAScript module documentation
- API security testing concepts
- OTP and MFA validation concepts
- WebSocket realtime testing concepts

3. IDENTIFICATION OF TESTABLE FEATURES

The Maal Wallet platform contains multiple automation-friendly features due to its API-driven architecture.

3.1 Critical Features

Feature	Importance
Health and Readiness	Ensures backend services are available
User Registration	Entry point for all users
Authentication Lifecycle	Protects user sessions
Device Binding	Prevents unauthorized access
PIN Reset and Recovery	Supports account security
Admin MFA	Protects privileged operations
Wallet Transactions	Core financial functionality
Peer-to-Peer Transfer	Main money transfer feature
Idempotency	Prevents duplicate transactions
Cash-In/Cash-Out	Agent-assisted financial operations
RBAC Validation	Prevents unauthorized actions

3.2 High Priority Features

Feature	Purpose
KYC Upgrade	Regulatory compliance
Transfer Limits	Rule enforcement

Disputes	Complaint handling
Admin User Management	Freeze/unfreeze operations
Risk Flags	Fraud monitoring
Notifications	User communication
Reports and Audit Logs	Compliance and traceability

3.3 Medium Priority Features

Feature	Purpose
Profile Management	User account maintenance
Agent Top-Up Requests	Agent liquidity management
Commission Rules	Financial rule configuration
System Configuration	Admin configuration management
BNPL Financing	Financing workflows
Dashboard Events	Monitoring and realtime updates

4. DEVELOPED TEST SCRIPTS

4.1 Test Suite Architecture

The automation suite is divided into multiple layers.

Layer	Purpose
Runner	Starts the framework and generates reports
Orchestrator	Executes flows based on dependencies
Context	Stores actors, tokens, fixtures, and results
Libraries	Provides reusable helpers
Reusable Blocks	Shared setup logic
Primary Flows	Main end-to-end scenarios
Standalone Probes	Independent endpoint checks
Negative Flows	Security and invalid scenarios
Validation Runner	Broad backend validation
Load Testing	WebSocket scalability testing

4.2 Flow Module Design

Each test flow contains:

- Flow ID
- Flow Name
- Dependencies
- Actors
- Endpoints
- Async execution steps

Example Flow

Attribute	Value
Flow ID	F-1
Name	User onboarding and wallet first-look

Dependencies F-26a

Actors U1

Endpoints Tested

- `POST /auth/register/request-otp`
- `POST /auth/register/verify-otp`
- `POST /auth/register/complete`
- `GET /me`
- `GET /me/wallet`

This modular structure improves scalability and maintainability.

4.3 Reusable Building Blocks

Common workflows are reused across scripts instead of duplicated.

Examples include:

- User registration
- Login
- Device binding
- MFA verification
- OTP verification

This improves maintainability and reduces repeated code.

4.4 HTTP Client and Assertions

The framework standardizes:

- Authentication handling
- Idempotency-key injection
- Logging

- Response validation
- Error handling

Assertion helpers validate:

- HTTP status codes
- Response body structure
- Error codes
- Business-rule correctness

4.5 Positive Scenario Example

Peer-to-Peer Transfer Flow

The transfer flow validates:

- User lookup
- Transfer creation
- Idempotency replay
- Transaction retrieval
- SMS notifications
- In-app notifications

This validates complete business behavior rather than only HTTP responses.

4.6 Negative Scenario Example

Idempotency Conflict Validation

The **N-12** script validates that:

- The same idempotency key cannot authorize different requests
- Duplicate financial actions are prevented

This protects transaction integrity.

4.7 Admin MFA Testing

Administrative operations require MFA verification.

The framework validates:

- MFA enrollment
- MFA verification
- Wrong MFA rejection
- Protected admin operations

This strengthens privileged-operation security testing.

4.8 Contract and Coverage Validation

The framework loads the OpenAPI contract and compares executed endpoints against the API specification.

This helps identify uncovered API operations.

4.9 Validation Runner

The validation runner performs broad backend verification including:

- Swagger discovery
- Health verification
- Authentication workflows
- Device binding
- Wallet transactions
- KYC
- Disputes
- BNPL
- Notifications
- Risk flags
- RBAC validation

4.10 Realtime Load Testing

The WebSocket load-testing module validates:

- Concurrent connections
- Connection failures
- Latency metrics
- Realtime message delivery
- Notification scalability

Metrics include:

- p50 latency
- p95 latency
- p99 latency

5. TEST EXECUTION AND RESULTS

This section is intentionally reserved for future execution results.

After execution, the following information should be included:

- Execution environment
- Commands used
- Total test count
- Passed tests
- Failed tests
- Blocked tests
- Coverage percentage
- Logs and screenshots
- Failure analysis
- Discovered defects

6. CI/CD PIPELINE INTEGRATION

6.1 Proposed CI/CD Workflow

The proposed CI/CD workflow includes:

1. Repository checkout
2. Dependency installation
3. Service startup
4. Environment-variable setup
5. E2E execution
6. Artifact upload
7. Coverage threshold enforcement

6.2 Pipeline Compatibility

The framework is designed to integrate with CI/CD systems such as:

- GitHub Actions
- GitLab CI/CD
- Jenkins
- Azure DevOps

The framework supports:

- Automated execution
- Structured logging
- Exit-code validation
- Artifact generation
- Coverage validation

7. REPORTING AND REFLECTION

7.1 Summary of Work

The automation framework was designed as a professional end-to-end API testing solution for financial systems.

The framework supports:

- Realistic business workflows
- Multi-actor validation
- Security testing
- Financial integrity validation
- Coverage tracking
- CI/CD integration

The major achievement is the creation of the:

Maal Flow-DAG Automation Method

7.2 Lessons Learned

Key lessons learned include:

- API testing must validate workflows, not only endpoints
- RBAC testing is critical for wallet systems
- Idempotency testing is essential in financial systems
- OTP and MFA flows require asynchronous validation
- Negative tests are as important as positive tests
- OpenAPI coverage improves automation completeness
- DAG execution simplifies complex dependencies

7.3 Challenges Encountered

Major challenges included:

- Stateful workflow management
- Multi-role coordination
- Async behavior validation
- Financial correctness validation
- Test-data isolation
- Security-rule enforcement
- Coverage versus reliability balance

7.4 Suggested Improvements

Future improvements include:

- Stronger balance-delta assertions
- Better error-code validation
- Audit-log verification
- Enhanced WebSocket payload validation
- Automatic report generation
- Expanded CI/CD reporting
- Frontend UI automation support

8. CONCLUSION

The Maal Wallet automation framework satisfies the core assignment requirements for:

- Framework selection
- Feature identification
- Automation-script development
- CI/CD planning
- Reflection and reporting

The custom Node.js framework is highly suitable for backend-driven wallet systems because it validates business-critical workflows directly at the API layer.

The Maal Flow-DAG Automation Method provides a scalable, maintainable, and professional automation structure that supports future regression testing and CI/CD integration.

Execution results and evidence should be added after the scripts are fully executed.